

SmartCook 手臂控制程式功能說明

對應檔案：`F60_F_左臂.as`（主切割臂） / `F60_R_右臂.as`（輔助固定臂） | 語言：Kawasaki AS Language

兩隻手臂各自跑一份幾乎對稱的程式，靠 `SYNC_STEP` 這組訊號交握，在每個動作階段互相等對方完成才進下一步。下面依「PC 怎麼下指令」「兩臂怎麼對齊」「內部各支程式做什麼」三層來說明。

一、PC 端可下達的指令（DISPATCH 分派表）

PC 透過 TCP 送一行 CSV 文字指令（例如 `PICKUP,PICKUP_CUCUMBER,F60_F`），控制器收到後由 `DISPATCH()` 依第一個欄位分派到對應的處理程式。

指令	參數	功能說明
<code>PICKUP</code>	位置, ARM	取料。兩臂協同移到指定取料點（如 <code>PICKUP_CUCUMBER</code> ），分「就緒 → 下降 → 集中 → 抬起」四階段，每階段都用 <code>SYNC_STEP</code> 跟另一隻手臂會合一次。 <code>ARM</code> 用來驗證這筆指令該由哪隻臂主導。
<code>CHOP</code>	食材, 切割次數, 切割厚度	切菜。F60_F 主導下壓切割，F60_R 配合壓住食材，一刀一刀重複到指定次數，每刀都要 <code>SYNC_STEP</code> 對齊。
<code>PLACE</code>	位置, 方式	放置食材。方式有 <code>POUR</code> （傾倒，需雙臂 <code>SYNC_STEP</code> 一起倒）、 <code>SCOOP</code> （舀取）、 <code>PUSH</code> （推）。
<code>FLIP</code>	次數, 速度百分比	翻炒。雙臂交替轉 90°/45°/135° 動作反覆幾個循環，模擬翻炒食材，靠 <code>SYNC_STEP</code> 逐階段對齊。
<code>HOME</code>	ARM	回到待機原點（ <code>HOME_LEFT</code> 或 <code>HOME_RIGHT</code> ）。
<code>STOP</code>	無	緊急停止： <code>BRAKE</code> 剎車、清掉訊號、忙碌狀態歸零。是唯一一個在手臂忙碌中仍會被執行的指令，其他指令這時候只會收到 <code>BUSY</code> 。
<code>RESET</code>	無	重置忙碌狀態、清訊號，讓手臂從卡住/錯誤狀態恢復成可接新指令。
<code>STATUS</code>	ARM	查詢目前狀態，回 <code>BUSY</code> 或 <code>OK</code> （可執行）。
<code>READY</code>	ARM	語意上是「這隻手臂準備好了嗎」，目前邏輯跟 <code>STATUS</code> 相同。
<code>IOTEST</code>	ON / OFF / READ	純 I/O 接線測試，直接開關/讀取雙臂交握訊號腳位，跳過 <code>SYNC_STEP</code> 與所有動作流程，用來單獨排查接線問題。
<code>HEARTBEAT</code>	無	心跳確認，回 <code>HEARTBEAT_ACK</code> ，在 <code>DISPATCH</code> 最前面單獨處理，不算在上面分派表內。
（其他任何字串）	—	回 <code>ERROR,E4021</code> （未知指令）。

二、雙臂同步機制

SYNC_STEP — 單階段會合

兩邊各自到位、完成一個階段的動作後呼叫 `SYNC_STEP`，用一組訊號（`sig_out_step` 輸出、`sig_in_step` 輸入）握手確認對方也做完了，才一起進下一階段。目前實作採「發起一回 pulse」設計，F60_R 是發起方、F60_F

是回應方：

檔案	角色	行為
F60_R_右臂.as 	發起方	拉高輸出 → 等 F60_F 回一個 pulse（收到 ON 或逾時都直接關輸出，不再等對方訊號變回 0）
F60_F_左臂.as 	回應方	等 F60_R 拉高 → 收到後回一個 0.1 秒的 pulse（拉高再拉低）→ 結束

逾時時間 `timeout_io_sec` 目前設為 30 秒；逾時會回 `ERROR,E4023` 給 PC。

底層訊號等待工具

函式	功能
<code>WAIT_SIGNAL</code>	等某個輸入訊號變 ON，逾時回傳失敗，是 SYNC_STEP 的基礎元件。
<code>WAIT_SIGNAL_OFF</code>	等某個輸入訊號變 OFF。目前 SYNC_STEP 新版設計已不使用，保留供之後參考。
<code>DO_IOTEST</code>	對應 IOTEST 指令的實作：ON 拉高、OFF 拉低、READ 回目前輸入訊號狀態（ <code>SIG,1</code> / <code>SIG,0</code> ）。

三、系統初始化

函式	功能
<code>INIT_SWITCHES</code>	設定控制器系統層級開關（連續軌跡、自動開機執行、訊息輸出等），兩臂設定完全相同，沿用原廠程式的既有設定。
<code>INIT_CONST</code>	設定本臂識別碼、TCP 通訊參數（埠號 9000 等）、雙臂交握訊號腳位編號、動作參數（趨近距離、切割深度等）、逾時秒數。
<code>INIT_POINTS</code>	宣告所有教點位置（取料點、切割區、放置區、翻炒用點位等）。多數點位仍是 <code>TRANS(0,0,0,0,0,0)</code> 佔位值，代表現場還沒教點；已教過的點位（如 F60_F 的 <code>ba_flip</code> / <code>ha_flip</code> ）會是實際座標。
<code>INIT_TOOL</code>	設定 BASE / TOOL 座標系與鏟具工具座標，供一般動作使用。
<code>heartput</code>	開機時先回到一個安全的關節角度姿態，避免從奇怪的殘留姿態直接動作。

四、TCP 通訊基礎架構

函式	功能
<code>MAIN</code>	整支程式的進入點：跑完初始化後進入「監聽 → 接受連線 → 收指令 → 分派 → 斷線後重新監聽」的無窮迴圈。
<code>CLEAN_SOCKET</code>	開機或重新監聽前，先清掉上次殘留的連線與監聽狀態，避免埠號卡住。
<code>OPEN_LISTEN</code>	開始在指定埠號監聽，失敗會重試並印出錯誤計數。
<code>WAIT_ACCEPT</code>	等待 PC 端連進來（ <code>TCP_ACCEPT</code> ）。
<code>DO_HANDSHAKE</code>	連線後的握手：收到 PC 送來的 <code>connect</code> 才回自己的 <code>BOARD_ID</code> 。

DISCONNECT	關閉目前的 socket 連線。
RECV_LINE	從 TCP 緩衝區組出一行完整指令（以換行字元 <code>\n</code> 分隔），處理跨封包接收。
SEND_LINE	送出一行回應（自動加上換行字元）。
SPLIT_CSV	把收到的一行文字依逗號切成欄位陣列，供 DISPATCH 判斷指令與參數。
DISPATCH	指令總機，依第一欄位決定呼叫哪個 <code>D0_*</code> 動作函式（見第一節表格）。

五、動作執行細節

函式	功能
D0_PICKUP	PICKUP 指令的實際動作序列：驗證主導臂 → 就緒 → 下降 → 集中收攏 → 抬起，每階段呼叫 SYNC_STEP。任一階段逾時就回 <code>ERROR,E4023</code> 並中止。
D0_CHOP	CHOP 指令的實際動作：依指定次數重複「下壓切割 → SYNC_STEP 對齊 → 提鏟 → 步進到下一刀」。
D0_PLACE	PLACE 指令的實際動作：移到目標位置，依方式做 <code>POUR</code> （傾倒，需 SYNC_STEP）或 <code>PUSH</code> （推）等後續動作。
D0_LTURN45 / D0_LTURN90 / D0_LTURN135 	F60_F 側的單次翻轉動作，分別轉 45° / 90° / 135°，各自搭配 SYNC_STEP 對齊。對應點位（ <code>lturn45_ready</code> 等）目前尚未教點。
D0_RTURN45 / D0_RTURN90 / D0_RTURN135 	F60_R 側對應的翻轉動作。對應點位（ <code>rturn45_ready</code> 等）與 <code>ba_flip</code> / <code>ha_flip</code> 目前尚未教點。
D0_FLIP	FLIP 指令的實際動作：切到翻炒專用 BASE/TOOL，依循環次數重複「90° 配對 → 135°/45° 配對 → 45°/135° 配對」，結束後切回原本的 BASE/TOOL。
D0_HOME	HOME 指令的實際動作：驗證是本臂的指令後，移回待機原點。
D0_STOP	STOP 指令的實際動作：剎車、清訊號、解除忙碌狀態。
D0_RESET	RESET 指令的實際動作：解除忙碌狀態、清訊號。
D0_STATUS / D0_READY	回報目前是否忙碌。

- 目前已知待處理事項：
- F60_R 的 `ba_flip` / `ha_flip` / `rturn45/90/135` 全部點位尚未教點，會導致 FLIP 動作出錯。
 - F60_F 的 `lturn45/90/135` 點位也尚未教點；`ba_flip` / `ha_flip` 已教過。
 - 多數 `PICKUP` / `PLACE` / `CHOP` 用到的點位（如 `PICKUP_CUCUMBER`、`WORK_CHOP_ZONE` 等）仍是佔位值，待 `CALIBRATION_POINTS.csv` 標定完成後填入。

SmartCook 展台自動化專案 | 本文件依 2026/08/03 控制器實際匯出的程式內容整理